

Absolutely. Below is the **Claude Code Development Specification** you can use after the client approves the project plan. It is based on the client's supplied requirements, while clearly separating features that are proposed for implementation from items the client still needs to confirm.

App Development

Dating App — Claude Code Development Specification

Document Version: 1.0

Purpose: Technical development specification

Development Tool: Claude Code

Target Market: India, Southeast Asia & Maldives

App Development

1. Product Vision

Develop a modern dating application focused on:

- Meaningful connections
- Profile authenticity
- Smart matching
- Location-based discovery
- Secure communication
- User safety
- Premium features
- Future AI-powered capabilities

The application should **not be a direct visual or functional copy of Tinder/Bumble**. It should have its own branding, UI/UX and product identity.

The client's requirements specifically recommend focusing on a distinct niche/community rather than competing directly with large dating platforms.

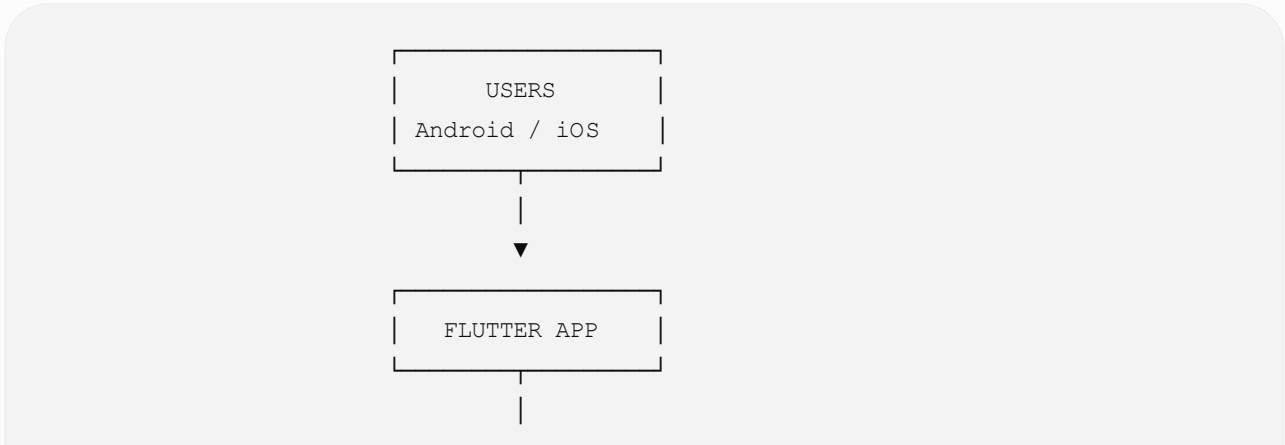
App Development

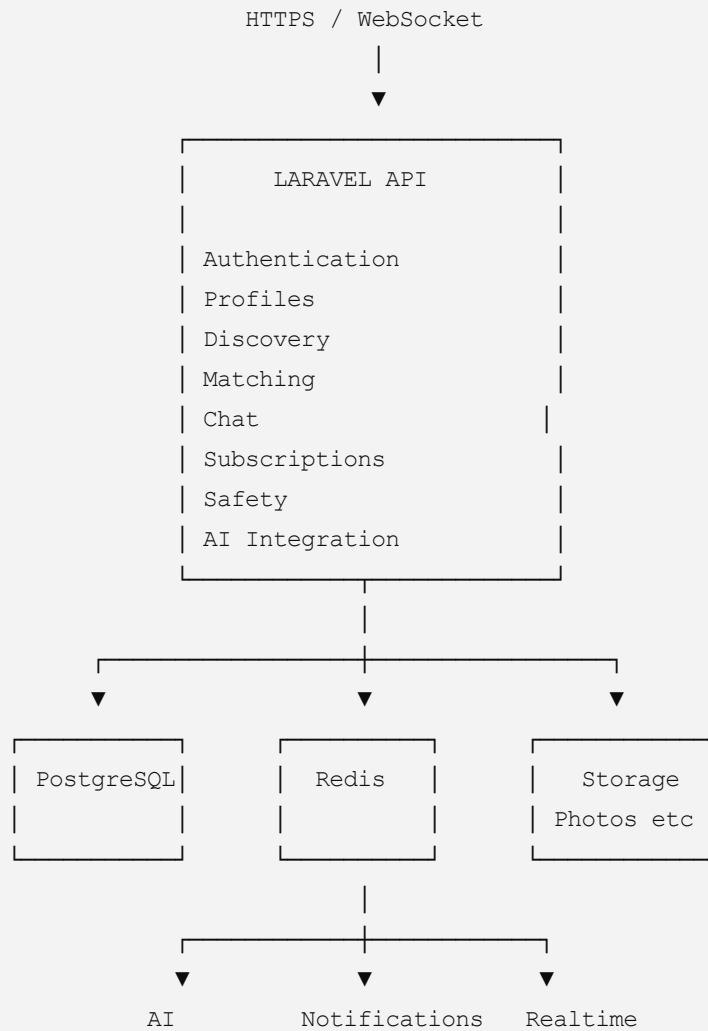
2. Technology Stack

Component	Technology
Mobile	Flutter
Backend	Laravel 12
Language	PHP 8.3+
Database	PostgreSQL

Component	Technology
Cache	Redis
API	REST API
Authentication	Laravel Sanctum + OAuth
Real-time	Laravel Reverb / WebSockets
Media	S3-compatible storage
Push Notifications	Firebase Cloud Messaging
AI	OpenAI API
Voice/Video	WebRTC / Agora
Maps	Google Maps / Mapbox
Admin	Laravel + Filament
Containerization	Docker
Version Control	GitHub
CI/CD	GitHub Actions
Monitoring	Sentry / application monitoring
Web Server	Nginx

3. System Architecture





4. Mobile Application

Flutter application modules:

```
lib/
├──
├── core/
├── authentication/
├── onboarding/
├── profile/
├── discovery/
├── matching/
├── chat/
├── notifications/
├── subscriptions/
├── verification/
├── calls/
├── safety/
└── settings/
```

Recommended architecture:

Feature-first + Clean Architecture principles

Each feature should contain its own:

```
presentation
domain
data
```

5. Authentication

Required

- Email login
- Phone number login
- OTP verification
- Google login
- Apple login

These are explicitly requested in the client's core features.

App Development

Additional

- Logout
- Forgot password
- Account deletion
- Session management
- Device management

6. Profile Module

Users should be able to create and edit:

- Profile photo
- Additional photos
- Name
- Age
- Bio
- Hobbies
- Interests
- Dating preferences
- Distance preference
- Relationship goals

The client specifically describes a basic profile engine containing photos, short bio and preferences such as age, distance and hobbies.

App Development

7. Profile Prompts

Add a dedicated prompt system.

Example:

I feel most like myself when...

Other examples:

- My ideal first date is...
- My simple pleasures are...
- Ask me about...
- A green flag I look for is...
- My perfect weekend is...

Users can select and answer multiple prompts.

Proposed database

```
profile_prompts
```

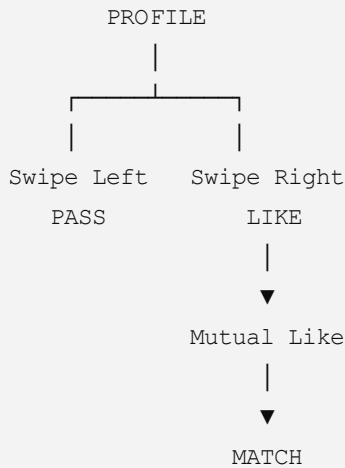
```
id
prompt
category
is_active
sort_order
created_at
updated_at
```

```
user_profile_prompts
```

```
id
user_id
prompt_id
answer
sort_order
created_at
updated_at
```

8. Discovery Module

Main experience:



Discovery filters

Initial:

- Age
- Distance
- Interests
- Relationship goals

Advanced filters:

- Religion
- Politics
- Additional preferences

Religion and politics are specifically listed as advanced filter requirements.

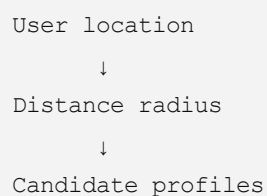
App Development

9. Geolocation

The application will use location to prioritize/restrict discovery based on a selected distance radius.

App Development

Example:



↓
Matching filters
↓
Discovery

Privacy

Do **not** expose:

- Exact coordinates
- Exact address
- GPS data

Display approximate information such as:

4 km away

10. Matching Engine

The matching engine should consider:

Age
Distance
Interests
Relationship Goals
Preferences
User Behaviour
Mutual Interests

The client's requirement specifically calls for matchmaking based on **behavior, preferences and mutual interests**.

App Development

Future compatibility score

Compatibility Score

- |
- ├─ Preferences
- ├─ Interests
- ├─ Behaviour
- ├─ Relationship Goals
- └─ Location

Example:

91% Compatible

The exact scoring formula should be finalized with the client before implementation.

11. Like / Pass

Core functionality:

- Like
- Pass
- Match
- Unmatch

Additional proposed features:

- Super Like
- Rewind

These should be treated as **proposed premium/engagement features**, not mandatory MVP requirements unless the client approves them.

12. Chat Module

Once users match:

- Real-time text chat
- Message history
- Read receipts
- Typing indicator
- Online/offline status
- Push notifications

The client specifically requires low-latency real-time text messaging.

App Development

Media communication

Client requirements also mention:

- Voice notes
- GIFs
- Photo sharing

App Development

These should be confirmed as MVP or Phase 2.

13. Unmatched Messaging

Special business rule:

Matched User



Can message normally

Unmatched User



Subscription required

The client's document explicitly states that sending a message to an unmatched user is restricted to subscription users. App Development

This rule must be enforced **on the backend**, not only hidden in the mobile UI.

14. Voice & Video Calling

Subscriber-only feature:

Match



Subscriber?



├ YES → Voice / Video



└ NO → Upgrade

The client explicitly specifies voice/video calling as subscriber-only. App Development

Technology can be finalized between:

- WebRTC
- Agora
- Another approved provider

15. Safety & Moderation

Mandatory:

- Block
- Report
- Unmatch
- Abuse reporting
- User moderation

The client's document identifies blocking/reporting as essential safety functionality.

App Development

Proposed report categories

- Harassment
- Fake profile
- Spam
- Inappropriate content
- Scam
- Other

16. Profile Verification

Proposed flow:

```
graph TD
    User --> VR[Verification Request]
    VR --> P[Photo / Selfie Prompt]
    P --> AV[AI Verification]
    AV --> VRs[Verification Result]
    VRs --> VB[Verified Badge]
```

The client's requirement specifically mentions AI/photo prompts for identity confirmation and reducing catfishing.

App Development

The exact verification provider and methodology should be confirmed before implementation.

17. Subscription System

Free

Potential features:

- Profile creation
- Discovery
- Basic filters
- Like/pass

- Matching
- Basic messaging

Premium

Potential features:

- Unlimited likes
- Advanced filters
- Profile boost
- Message unmatched users
- Voice calling
- Video calling

The client specifically mentions subscriptions offering profile boosts, unlimited likes and advanced filters while keeping basic messaging available.

App Development

Pricing and exact entitlements must be confirmed by the client.

18. Gamification

Possible features:

- Profile prompts
- Badges
- Swipe mechanics
- Profile completion
- Engagement achievements

The client's requirements specifically mention badges, prompts and swipe mechanics as engagement mechanisms.

App Development

19. Notification System

Notifications for:

- New match
- New message
- Like
- Subscription
- Verification
- Report status
- System announcements

Technology:

Firebase Cloud Messaging

20. Admin Panel

Dashboard

Total Users
Active Users
New Users
Matches
Messages
Reports
Premium Users
Revenue

User Management

- Search
- View
- Suspend
- Ban
- Delete

Moderation

- Reports
- Verification
- Suspicious accounts
- Content moderation

Subscription

- Plans
 - Users
 - Payments
 - Subscription status
-

21. Database Architecture

Core tables:

```
users
profiles
profile_photos
profile_prompts
user_profile_prompts

interests
user_interests
user_preferences

user_locations

swipes
likes
matches

conversations
messages
message_attachments

blocks
reports

subscriptions
subscription_plans
payments
boosts

notifications

verification_requests
verification_results

user_devices
```

Future AI tables:

```
ai_profile_analysis
compatibility_scores
recommendation_history
```

22. API Architecture

Example:

```
/api/v1/auth
/api/v1/users
```

```
/api/v1/profiles  
/api/v1/interests  
/api/v1/prompts  
/api/v1/discovery  
/api/v1/swipes  
/api/v1/matches  
/api/v1/chat  
/api/v1/notifications  
/api/v1/subscriptions  
/api/v1/payments  
/api/v1/verification  
/api/v1/calls  
/api/v1/safety
```

All APIs should use:

- Authentication
 - Authorization
 - Request validation
 - API resources
 - Rate limiting
 - Consistent error responses
-

23. Development Phases

Phase 0 — Architecture & Design

- Requirements confirmation
- UI/UX
- Database design
- API specification
- Security architecture

Phase 1 — MVP

- Authentication
- Onboarding
- Profile
- Photos
- Prompts
- Discovery
- Like/Pass
- Matching
- Real-time chat
- Block/Report

Phase 2 — Premium

- Subscription
- Advanced filters
- Unlimited likes
- Boost
- Unmatched messaging
- Premium management

Phase 3 — Communication

- Voice notes
- GIFs
- Photo sharing
- Voice calls
- Video calls

Phase 4 — AI

- AI profile verification
- Compatibility scoring
- Smart recommendations
- AI icebreakers

Phase 5 — Production

- Security audit
- Performance testing
- App Store preparation
- Play Store preparation
- Production deployment
- Monitoring
- Backup
- Analytics

24. Claude Code Development Rules

Claude Code should **not** build the entire application in one operation.

Use:

Requirement

↓

Architecture

↓

Feature specification
↓
Implementation
↓
Tests
↓
Code review
↓
Fix
↓
Git commit

For every feature:

Implement → Test → Review → Fix → Commit

Claude should not move to the next phase until the current phase passes its tests.

25. Estimated Timeline

For a properly developed production application, I would initially propose:

Phase	Estimated Duration
Requirements + UI/UX	2–3 weeks
Architecture + Database	1–2 weeks
MVP Development	6–8 weeks
Premium Features	3–4 weeks
Communication/Calls	2–3 weeks
AI Features	3–4 weeks
Testing & Security	2–3 weeks
Deployment	1 week
Total	20–28 weeks

This is an **initial planning estimate**, not a fixed commitment. Final duration should be determined after the client confirms the exact scope, UI/UX, integrations and subscription requirements.

26. Client Decisions Required Before Development

There are several things I would send to the client for confirmation:

Product

1. Exact target niche/community
2. Brand name
3. Countries for initial launch
4. Age restrictions
5. Relationship categories

Matching

6. Gender options
7. Age range
8. Distance range
9. Matching criteria
10. Advanced filters

Premium

11. Number of subscription plans
12. Pricing
13. Premium benefits
14. Boost system
15. Unmatched messaging rules

Communication

16. Voice notes?
17. GIFs?
18. Photo sharing?
19. Voice calls?
20. Video calls?

Verification

21. AI verification?
22. Manual verification?
23. ID verification?

AI

24. AI compatibility score?
25. AI icebreakers?
26. AI profile assistance?

Business

- 27. Payment gateway
 - 28. App Store / Google Play subscription strategy
 - 29. Admin users
 - 30. Launch date
-

What I recommend you send the client

Don't send the **Claude Code section** to the client. That's your internal development methodology.

For the client, make the document:

“Dating Application — Technology & Architecture Proposal”

with:

- 1. Project Overview
- 2. Objectives
- 3. Target Market
- 4. Functional Scope
- 5. Technology Stack
- 6. System Architecture
- 7. Security Architecture
- 8. Development Phases
- 9. Timeline
- 10. Deliverables
- 11. Client Responsibilities
- 12. Assumptions
- 13. Exclusions
- 14. Commercials
- 15. Approval

Once they approve that, **we use this technical specification internally to drive Claude Code development.**

If you want, I can next create the **actual polished client-facing proposal as a PDF**, with a professional architecture diagram, technology-stack table, phases, timeline, deliverables, assumptions and exclusions, ready to send to the client.

